

Android Native DevCamp Handbook

Android Native DevCamp

Overview

Welcome future Android Commandos! This DevCamp will take you on a journey through the world of native Android development. From the basics of Kotlin and Android through to building, signing, and distributing your own app — all delivered as five focused milestones.

Duration

- 4-6 weeks

Prerequisites

- General programming skills
- Familiarity with MVVM (optional, but beneficial)

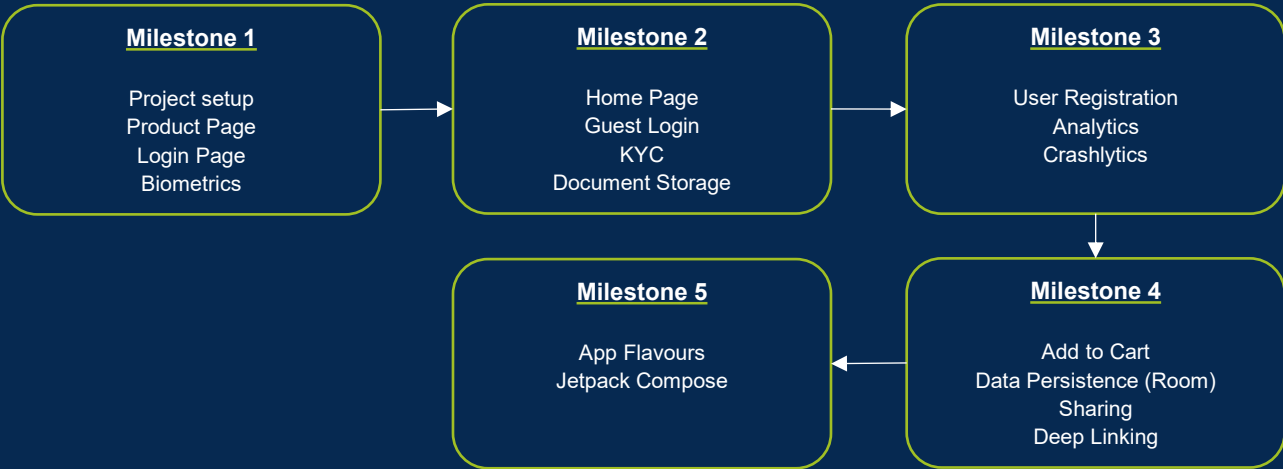
Technical Requirements

- Android Studio
- Android phone (optional, but beneficial)

Concepts covered

- | | | |
|---|--|---|
| <ul style="list-style-type: none">• Kotlin• Android Studio• XML based views (imperative UI) | <ul style="list-style-type: none">• Jetpack Compose (declarative UI)• Firebase• App Flavours | <ul style="list-style-type: none">• Modularization• All things Android• Signed APK distribution |
|---|--|---|

Timeline



Getting started

Setting up your Development environment

In order to get going on your Android development journey, let's first get your development environment set up correctly.

1. Install Android Studio

Although there are a few different IDEs which can be used for Android development, Android Studio is the preferred IDE. It is also beneficial if the entire team works with the same tools. Download the latest version of [Android Studio from here](#).

2. Create a Google Account with Entelect email address (optional)

Create a new Google account using your Entelect email address (use a different browser than the one where you might already be signed in to Google, or a private/incognito window to prevent mixing Google accounts). During the account creation, select personal account and select the checkbox for using your own email address. You'll receive an email with a verification code to confirm your email.

3. API

The DevCamp mobile app will be developed against a provided API. More details to be added here soon. Please refer to Appendix A for info about the API if you wish to host your own instance locally (not recommended).

4. Git

We'll be using Git for version control. If not already installed, follow the instructions on the [official Git website](#) or install your favourite [Git GUI](#). Everyone will work together in a single provided repository which will also contain a seed project.

5. Physical device (optional)

An emulator is great for developing against, however we found that using a physical Android device provides an overall better experience. Please note that in the case of hosting your own API rather than the provided API you will need to access it from your phone using something like ngrok.

Next Steps

You're now equipped with the essential tools and resources needed for the Android DevCamp. Let's get going!

The guide is organized into five numbered milestones, each focusing on specific Android development concepts and tasks. Start with Milestone 1 and progress through each milestone sequentially.

You'll find more details on the next page.

Need Help?

If you encounter any challenges or have questions, reach out to your Team Lead. Join the DevCamp Teams channel where you can connect with fellow engineers for support.

Now, let the coding adventure begin! Happy coding, and enjoy the journey of becoming an Android developer.

Milestone breakdown

Milestone 1

Project setup
Product Page
Login Page
Biometrics

Milestone 2

Home Page
Guest Login
KYC
Document Storage

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 1

Deliverables

- **Project setup**

Create your Android project and push it to your repository.

Create a Firebase project, add it as a dependency in your Android project and complete the Firebase configuration.

- **Implement first screen**

Build the product page according to the designs and use it as your initial screen. On the Figma designs, focus on the "guest flow" – thus no need to implement login. Make sure to use the XML view based UI framework and NOT jetpack compose (more on jetpack compose later).

- **Distribute app for testing**

Create a release build and upload it to Google Play. You should be able to download it via the Play Store.

Resources

- **BRS requirement**

US4 – View Products

- **Designs**

[Figma](#) (Ready for Development – Home)

Useful Links

The [Android docs](#) is a goldmine of information. Be sure to refer to it regularly.

Project

- [General Android Studio info](#)
- [Project structure](#)
- [Logcat](#)
- [Debugging](#)
- [Manifest](#)
- [Gradle Build](#)

Basics

- [Kotlin Basics](#)
- [Activity Lifecycle](#)
- [Fragment Lifecycle](#)
- [Full course to reference](#)

UI

- [Layouts and Views](#)
- [Dynamic Lists](#)
- [MVVM in 100 seconds](#)
- [UI Layer](#)
- [View binding](#)

Milestone breakdown

Milestone 1

Project setup
Product Page
Login Page
Biometrics

Milestone 2

Home Page
Guest Login
KYC
Document Storage

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 1 (cont.)

Deliverables

- **Login page**

Build a secure Login Page by following the designs. Use the provided auth service.

- **Biometrics**

Add support for biometric authentication.

Resources

- **BRS requirement**

US1 – Login

- **Designs**

[Figma](#) (Ready for Development – Login)

Useful Links

Basics

- [Navigation](#)
- [Backstack](#)

UI

- [View Models](#)
- [Koin Viewmodel](#)
- [Livedata](#)
- [Theming](#)

Networking

- [Retrofit](#) with [Co-Routines](#)

Data storage

- [SharedPreferences](#)
- [Encrypted SharedPreferences](#)

Biometrics

- [Android docs](#)
- [Things to consider](#)

Milestone breakdown

Milestone 1

Project setup
Product Page
Login Page
Biometrics

Milestone 2

Home Page
Guest Login
KYC
Document Storage

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 2

Deliverables

- **Home Page**
Build the Home page to display available products to the user. Include loading and error screens. Use the provided Product Catalogue service.
- **Guest Login**
Allow anyone to access the app. Only certain products are available and some functionality is blocked. See designs and BRS for details.

Resources

- **BRS requirement**
US4 – View Products
- **Designs**
[Figma](#) (Ready for Development – Home)

Useful Links

Basics

- [Modularization](#)
- [Modularization patterns](#)
- [Multi-module navigation](#)

UI

- [View Models](#)
- [Koin Viewmodel](#)
- [Livedata](#)

Firebase

- [Remote config](#)

Milestone breakdown

Milestone 1

Project setup
Product Page
Login Page
Biometrics

Milestone 2

Home Page
Guest Login
KYC
Document Storage

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 2 (cont.)

Deliverables

- **Know Your Customer**
Implement a KYC flow where the user has to upload a proof of residence and a selfie. Allow the user to take a photo or select a file.
- **Document Storage**
Use Firebase to store the uploaded documents.

Resources

- **BRS requirement**
US7 – Do Customer Checks
- **Designs**
[Figma](#) (Ready for Development – Registration – [KYC Flow](#))
NOTE: Not the whole registration flow, only the KYC/Identity Verification Flow.

Useful Links

Basics

- [Permissions](#)
- [Permissions considerations](#)
- [Data storage overview](#)

Document Selection

- [Document selection](#)
- [Limited document types](#)

Firebase

- [Storage introduction](#)
- [Storage getting started](#)

Camera

- [Basic camera intent](#)
- [CameraX library](#)
- [CameraX codelab](#)

Photo Selection

- [Photo picker](#)

Milestone breakdown

Milestone 2

Home Page
Guest Login
KYC
Document Storage

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 4

Add to Cart
Data Persistence
Sharing
Deep Linking

Milestone 3

Deliverables

- **User Registration**

Build a user registration flow with input validation. Include the completed KYC flow as a step.

- **Analytics**

Use Firebase to add analytics to the app. Track when a user drops out of registration. Optionally add Crashlytics.

Resources

- **BRS requirement**

US2 – Create Profile

- **Designs**

[Figma](#) (Ready for Development – Registration – [Registration Flow](#))

Useful Links

Analytics

- [Logging events](#)
- [Screenview Events](#)
- [Analytics Dashboard](#)

Crashlytics

- [Crashlytics basics](#)

Dialogs

- [Dialog Fragment](#)
- [Bottom Sheet](#)

Milestone breakdown

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 4

Add to Cart
Data Persistence
Sharing
Deep Linking

Milestone 5

App Flavours
Jetpack Compose

Milestone 4

Deliverables

- **Add to Cart**

Allow the user to add items to their cart and check out. Users need to be logged in to complete the process.

- **Data Persistence**

When a user adds items to their cart, those items should be there after closing and re-opening the app.

Resources

- **BRS requirement**

US8 – Generate contract

- **Designs**

[Figma](#) (Ready for Development – [Checkout](#))

Useful Links

Storage

- [Room](#)
- [Room Entities](#)
- [Room DAOs](#)

Milestone breakdown

Milestone 3

User Registration
Analytics
Crashlytics

Milestone 4

Add to Cart
Data Persistence
Sharing
Deep Linking

Milestone 5

App Flavours
Jetpack Compose

Milestone 4 (cont.)

Deliverables

- **Sharing**

Allow the user to share products with friends over social platforms. Ensure that a link back to the app is included. Optionally allow the user to download a product's details as a PDF.

- **Deep Linking**

Open your app when a user taps on a link on their device.

Resources

- **BRS requirement**

None

- **Designs**

Figma (Ready for Development – Home – [Product Enhancement](#))

Useful Links

Deep Linking

- [App links](#) (understand the different types of links)
- [Deep link to App Content](#)
- [Deep Linking navigation](#)

Document Selection

- [Document selection](#)
- [Limited document types](#)

Camera

- [Basic camera intent](#)
- [CameraX library](#)
- [CameraX codelab](#)

Photo Selection

- [Photo picker](#)

Milestone breakdown

Milestone 3

User Registration
Analytics
Crashlytics



Milestone 4

Add to Cart
Data Persistence
Sharing
Deep Linking



Milestone 5

App Flavours
Jetpack Compose

Milestone 5

Deliverables

- **App Flavours**

Allow the app to be built for multiple environments. Change the colour scheme per environment. Allow features to be disabled for certain environments.

- **Declarative UI**

Refactor a screen of your selection using Jetpack Compose.

Resources

- **BRS requirement**

None

- **Designs**

Figma ([Colors](#))

Useful Links

White labelling

- [Build variants & App Flavours](#)
- [Styles & Themes](#)

Declarative UI

- [Jetpack compose](#)
- [Thinking compose](#)
- [Layouts](#)
- [Migrate from Views to Jetpack Compose](#)

Appendix

Appendix A

Setting up API locally

Please note that it is recommended to rather develop against the hosted API.

If you wish to set up the API against which you'll develop the mobile app locally, then follow these instructions. You will need:

- Docker running in WSL2 with a Linux Distro (Docker Desktop should not be used due to licensing requirements)
- [Bruno](#) to test/explore the running API

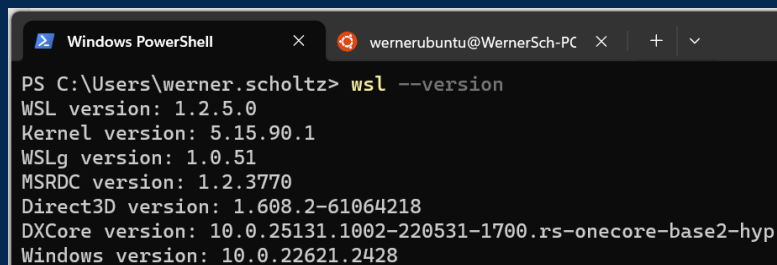
WSL2

References: [Guide to installing wsl](#)

Ensure wsl is installed by running the below command in powershell:

`wsl --version`

The above command should return something like below if WSL is installed:



```
Windows PowerShell
PS C:\Users\werner.scholtz> wsl --version
WSL version: 1.2.5.0
Kernel version: 5.15.90.1
WSLg version: 1.0.51
MSRDC version: 1.2.3770
Direct3D version: 1.608.2-61064218
DXCore version: 10.0.25131.1002-220531-1700.rs-onecore-base2-hyp
Windows version: 10.0.22621.2428
```

If WSL is not installed, install it with the below command (which will also install Ubuntu):

`wsl --install`

LINUX

Once WSL is installed, check if you have a linux distro installed. Either just search for the distro (ex. Ubuntu) in windows search, or in Terminal select the dropdown where the linux distro should be an option to choose from next to other terminal windows like Powershell and Command Prompt.

If Linux is not installed, then either install it from the Microsoft Store (ex. Ubuntu 22.04.02 LTS), or from command line using:

`wsl --install -d ubuntu`

Once the install is done and you open ex. Ubuntu, go through the setup where you'll likely need to choose a username and password.

Appendix A

Setting up docker (cont.)

DOCKER

References: [Installing docker](#)

Once Linux is running you need to install docker.

The following is all from within the Linux scope. Thus from within the Linux terminal, not from Powershell.

Make sure that you can connect to the update servers by attempting an update:

```
sudo apt-get update
```

If it does not complete successfully, then try running the below command which should sort it out:

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf > /dev/null
```

Once you are sure you can connect to the update servers, then continue following the instructions to install docker using the apt repository (it's mostly copy pasting commands):

<https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>

The gist of the above link is that you will 1. Set up Docker's apt repository access, and 2. Install the docker packages.

Once done with step 2 in the above link/instructions, check if docker is working by running the following command which should print out any running docker containers:

```
sudo docker ps
```

It should print out some headings, but there will likely not be any containers running yet. If it does give an error, try restarting the Linux instance (maybe just restart your PC) and try again.

DevCamp Services

Now that you have an instance of docker running on your PC we can run the API's images as containers.

The API details can be found [here](#). The API and it's corresponding database is provided as docker images. If you don't have permission to access this repo, then ask for assistance. Clone the repo. You'll need the API collection and "compose.yaml" file. Be sure to read through the README and follow the instructions in order to set up pull access from Entelect's docker repo called "Harbor",

The compose.yaml file is what we'll use to start up the containers. The easiest way to do this is to navigate to the compose.yaml file using File Explorer, then right click and select "Open Linux shell here". Once the Linux shell has opened, run the compose file with the following command:

```
sudo docker compose up -d
```

You might be prompted to install docker compose first. Once the "docker compose" command executed successfully, check if the containers are running with:

```
sudo docker ps
```

There should be 2 containers running. One for the PostGres DB and one for the API.